

# Failure (a dead connection, and a real container)

## — Hints

---

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper error handling, a real Dockerfile, and proof that no secret leaked). Read Basic first even if you've used Docker before — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 1 — The idea, and the exact pieces

### Basic Version

This exercise is really two separate things: breaking your database connection on purpose to prove your error handling actually works, and packaging your whole app into a container that runs the same way anywhere.

Things to use for the database part:

- Point your `sqlite3.connect(...)` call at a folder that doesn't exist, like `"/no/such/path/runs.db"`.
- Hit any route that touches the database, and check you get a `500` back — not a crash, not a raw Python traceback shown to the client.

Things to use for the Docker part:

- A `Dockerfile` with 4 kinds of lines: `FROM`, `COPY` + `RUN` (install), `COPY` (your code), `CMD` (how to start it).
- `docker build -t my-api .`
- `docker run -p 8000:8000 -e OPENAI_API_KEY=sk-... my-api`

### Intermediate Version

A broken database connection should be caught by the same `try/except` pattern `chat_route` already built — the difference here is proving it, on purpose, against something real (a genuinely bad path), not a fake exception you raise by hand. Note that `sqlite3.connect()` itself can fail too, so the connect call needs its own guard — otherwise the whole app fails to start.

For the Dockerfile, the *order* of the lines matters: `COPY requirements.txt .` and `RUN pip install -r requirements.txt` go **before** `COPY . .`. Docker saves each line as a layer; if `requirements.txt` hasn't changed, Docker reuses the saved install step instead of re-running `pip install` on every build.

Anything `COPY`-ed into an image stays in one of its layers for good — even if a later line deletes it. So a `.env` file must never be copied in. A `.dockerignore` file stops `COPY . .` from picking it up by accident, and `docker history --no-trunc my-api` lets you check every layer yourself.

The exact pieces:

- `sqlite3.connect("/no/such/path/runs.db")` raises `sqlite3.OperationalError: unable to open database file` (SQLite doesn't create missing folders). Wrap the connect in `try/except sqlite3.OperationalError:` and set `conn = None`, so the app still starts and only database routes fail.
- A Dockerfile (the `\` just continues the `CMD` line onto a second line): `FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install -r requirements.txt COPY . . CMD ["uvicorn", "db_failure_and_docker_practice:app", \ "--host", "0.0.0.0", "--port", "8000"]`
- `--host 0.0.0.0` matters inside a container — `127.0.0.1` (uvicorn's default) only accepts connections from *inside* the container, so your `curl` would never reach it.
- `.dockerignore`: `.env`, `.git`, `__pycache__`/, `*.pyc`, `.venv/`.
- `docker run -e KEY=value` passes an environment variable in at run time — never baked into the image. Then `docker history --no-trunc my-api | grep "sk-test-123"` must print nothing.
- A required env var that's missing at container start should fail loudly and at once, not hang — Doc01's startup check: `os.getenv(...)`, and if it's empty, raise `MissingConfigError` with a clear message.

**Difference between Basic and Intermediate:** Basic names the pieces for both halves — breaking a connection, and a minimal Dockerfile. Intermediate guards the connect call itself so the app still starts, explains why the Dockerfile's line order matters, why `--host 0.0.0.0` is needed in a container, how `.dockerignore` and `docker history` keep and prove secrets out of the image, and why a missing required setting should stop the container at startup.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Hint 2 — The plan, and almost the whole thing

### Basic Version

```
temporarily change the db path to somewhere that doesn't exist
hit a route that touches the database
check: you get 500, not a crash, not a raw traceback

write a Dockerfile:
    start from python:3.11-slim
    copy requirements.txt, install it
    copy the rest of the code
    say how to start the app

docker build -t my-api .
docker run -p 8000:8000 -e OPENAI_API_KEY=sk-... my-api
curl localhost:8000/health # should work the same as running it locally
```

Here's almost the whole thing for the database half:

```
# practice/db_failure_and_docker_practice.py
import sqlite3
from fastapi import FastAPI, HTTPException

app = FastAPI()
DB_PATH = "/no/such/path/runs.db" # broken on purpose

@app.get("/runs")
def get_runs():
    try:
        # connect inside the try: connect() itself fails on a bad path
        conn = sqlite3.connect(DB_PATH)
        rows = conn.execute("SELECT * FROM runs").fetchall()
        return rows
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

**Expected output ( `curl -i localhost:8000/runs` ):**

```
HTTP/1.1 500 Internal Server Error
{"detail":"Something went wrong."}
```

And here's almost the whole Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "db_failure_and_docker_practice:app", \
    "--host", "0.0.0.0", "--port", "8000"]
```

## Intermediate Version

```
guard the connect + CREATE TABLE with try/except sqlite3.OperationalError:
    on failure, conn = None – the app still starts
route GET /runs:
    if conn is None: 500
    try the query, 500 on any error
```

at the very top: read OPENAI\_API\_KEY; if it's empty, raise  
MissingConfigError with a clear message (Doc01)

Dockerfile as in Basic, plus a .dockerignore (.env, .git, \_\_pycache\_\_, venvs)

```
after building and running:
docker run -e OPENAI_API_KEY=sk-test-123 -p 8000:8000 my-api
curl localhost:8000/health -> same output as running locally
docker history --no-trunc my-api | grep "sk-test-123" -> nothing
docker run my-api (no -e) -> exits at once with a clear error
```

Here's most of it — fill in the required-env-var check yourself:

```
# practice/db_failure_and_docker_practice.py
import os
import sqlite3
from fastapi import FastAPI, HTTPException

class MissingConfigError(Exception):
    """Same custom error as Doc01's exceptions.py."""

# your turn: read OPENAI_API_KEY with os.getenv(...); if it's
```

```
# missing or empty, raise MissingConfigError with a clear message

app = FastAPI()

try:
    conn = sqlite3.connect("/no/such/path/runs.db", check_same_thread=False)
    conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY)")
except sqlite3.OperationalError:
    conn = None # app still starts, but routes using it fail cleanly

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    if conn is None:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)